

Into the woods: Finding your way with **B-trees** in PostgreSQL

Sergey Dudoladov



Who am I ?

Senior Database Engineer

Organizer of PG events

Zalando SE

~ 3000 PostgreSQL clusters

Open source: PG operator and Spilo

Who are you ?

**Developers who want to
understand**

Why this series ?

PostgreSQL is fun

Previous talk

Lifetime of a SELECT

<https://sllnk.com/tc96xyz>

Philosophy

More why then how

Simple (not easy)

**All models are wrong but some are
useful**

George E.P. Box, a British statistician

Technicalities

server_version

18.3 (Ubuntu 18.3-1.pgdg22.04+1)

set max_parallel_workers_per_gather = 0;

AI Policy

**No GPTation
without
experimentation**

The Plan

The Almighty Tree

The Concept of Order

The Curse of Dimensionality

A database **index** is a data structure
that improves the speed of data
retrieval operations

Wikipedia

Full scan: $\sim n$

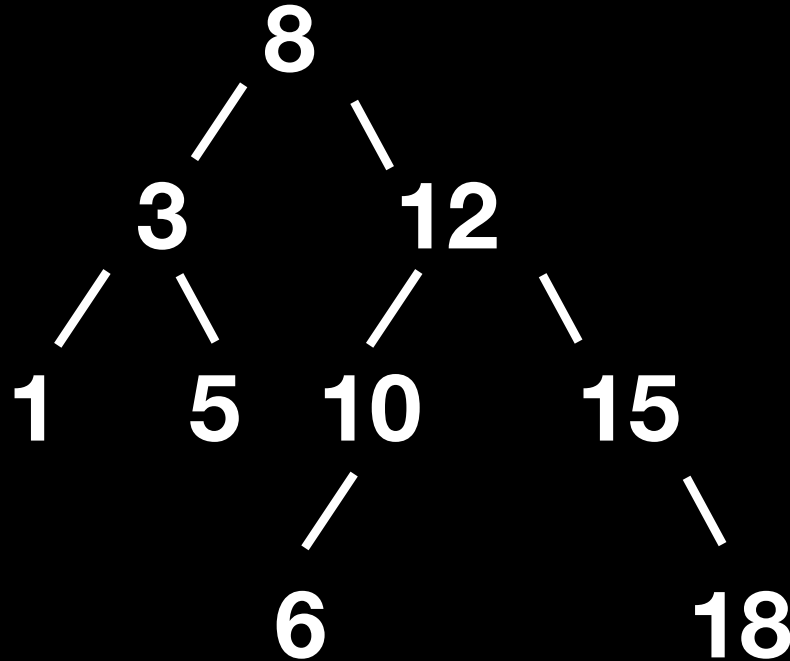
8, 3, 15, 1, 6, 12, 18, 5, 10

Binary search: $\sim \log n$

1, 3, 5, 6, 8, 10, 12, 15, 18

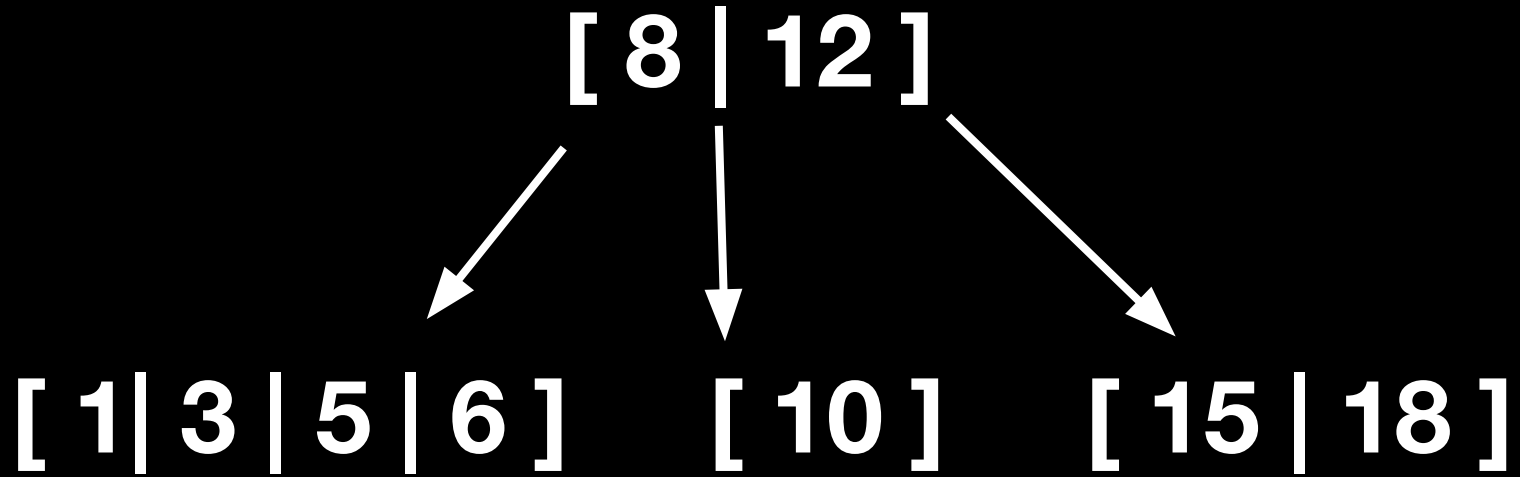
Order enables speed

Materialize binary search



IO happens in pages

The B-tree



B-trees are shallow

Levels 4

Size 14 GB

Pages 1_889_024

Tuples 675_549_000

```
CREATE EXTENSION pageinspect;  
SELECT  
    c.relname,  
    pg_size_pretty(pg_relation_size(c.oid)) AS size,  
    c.relpages,  
    c.reltuples::numeric(20,0) AS reltuples,  
    (SELECT level + 1 FROM public.bt_metap(c.relname)) AS levels  
FROM pg_class c  
WHERE c.relname = 'your_index' \gx
```

The Naming Mystery

A **B**ushy **B**alanced tree
co-invented by Rudolf **B**ayer
In **B**oeing Research Labs

The Plan

The Almighty Tree

The Concept of Order

The Curse of Dimensionality

Well-defined linear order

Any two elements are comparable

The “I don’t know” value

<	42	Null
42	False	Null
Null	Null	Null

You decide !

42 Null

Null 42

```
DROP TABLE IF EXISTS t_null_demo;  
CREATE TABLE t_null_demo (  
    x INT  
);  
-- 2M rows, ~20% NULLs  
INSERT INTO t_null_demo (x)  
SELECT  
    CASE  
        WHEN random() < 0.2 THEN NULL  
        ELSE (random() * 1000)::int  
    END  
FROM generate_series(1, 2_000_000);  
CREATE INDEX idx_x_nulls_last ON t_null_demo (x ASC NULLS LAST);  
ANALYZE t_null_demo;
```

```
EXPLAIN ANALYZE SELECT x FROM t_null_demo ORDER BY x ASC NULLS LAST LIMIT 10;  
Limit (cost=0.43..0.61 rows=10 width=4) (actual time=0.027..0.030  
rows=10.00 loops=1)  
  Buffers: shared hit=4  
    -> Index Only Scan using idx_x_nulls_last on t_null_demo  
        (cost=0.43..37012.43 rows=2000000 width=4) (actual time=0.025..0.027  
rows=10.0)  
      Heap Fetches: 0  
      Index Searches: 1  
      Buffers: shared hit=4  
Planning Time: 0.089 ms  
Execution Time: 0.048 ms  
(8 rows)
```

```
EXPLAIN ANALYZE SELECT x FROM t_null_demo ORDER BY x NULLS FIRST LIMIT 10;
Limit (cost=71658.28..71658.31 rows=10 width=4) (actual time=222.785..222.788
rows=10.00 loops=1)
  Buffers: shared hit=8439
    -> Sort (cost=71658.28..76658.28 rows=2000000 width=4) (actual
time=222.783..222.784 rows=10.00 loops=1)
      Sort Key: x NULLS FIRST
      Sort Method: top-N heapsort  Memory: 25kB
      Buffers: shared hit=8439
        -> Seq Scan on t_null_demo (cost=0.00..28439.00 rows=2000000
width=4) (actual time=0.016..104.553 rows=2000000.00 loops=1)
          Buffers: shared hit=8439
Planning Time: 0.090 ms
Execution Time: 222.807 ms
(10 rows)
```

Index defines how NULLs are sorted

**If your query expects a different order,
PostgreSQL will help**

The tale of two columns



```
DROP TABLE IF EXISTS multi_col_demo;  
CREATE TABLE multi_col_demo (  
    x INT,  
    y INT  
);  
-- 5M rows, x has 10 distinct values, y is unique  
INSERT INTO multi_col_demo (x, y)  
SELECT  
    g % 10,  
    g  
FROM generate_series(1, 5_000_000) AS g;  
CREATE INDEX idx_xy ON multi_col_demo (x, y);  
ANALYZE multi_col_demo;
```

```
EXPLAIN ANALYZE SELECT * FROM multi_col_demo WHERE x = 3 ORDER BY y LIMIT 10;
Limit (cost=0.43..0.72 rows=10 width=8) (actual time=0.060..0.066 rows=10.00
loops=1)
  Buffers: shared hit=4
    -> Index Only Scan using idx_xy on multi_col_demo (cost=0.43..14003.51
rows=491833 width=8) (actual time=0.057..0.060 rows=10.00 loops=1)
      Index Cond: (x = 3)
      Heap Fetches: 0
      Index Searches: 1
      Buffers: shared hit=4
Planning Time: 0.138 ms
Execution Time: 0.095 ms
```

```
EXPLAIN ANALYZE SELECT * FROM multi_col_demo WHERE y = 123453;
```

```
Seq Scan on multi_col_demo (cost=0.00..84624.00 rows=1 width=8)
```

```
(actual time=31.527..709.154 rows=1 loops=1)
```

```
Filter: (y = 123453)
```

```
Rows Removed by Filter: 4999999
```

```
Planning Time: 0.066 ms
```

```
Execution Time: 709.181 ms
```



**A phone book is ordered by last name,
but you only know the first name**

Relax, it was PG 17.9

PG 18 introduces skip scans

**For every last name, search efficiently
for the first name ***

*** Assume you have few last names**

```
EXPLAIN ANALYZE SELECT * FROM multi_col_demo WHERE y = 123453;
```

```
Index Only Scan using idx_xy on multi_col_demo (cost=0.43..48.85  
rows=1 width=8) (actual time=0.304..0.445 rows=1.00 loops=1)
```

```
Index Cond: (y = 123453)
```

```
Heap Fetches: 0
```

```
Index Searches: 12
```

```
Buffers: shared hit=15 read=22
```

```
Planning Time: 0.073 ms
```

```
Execution Time: 0.463 ms
```

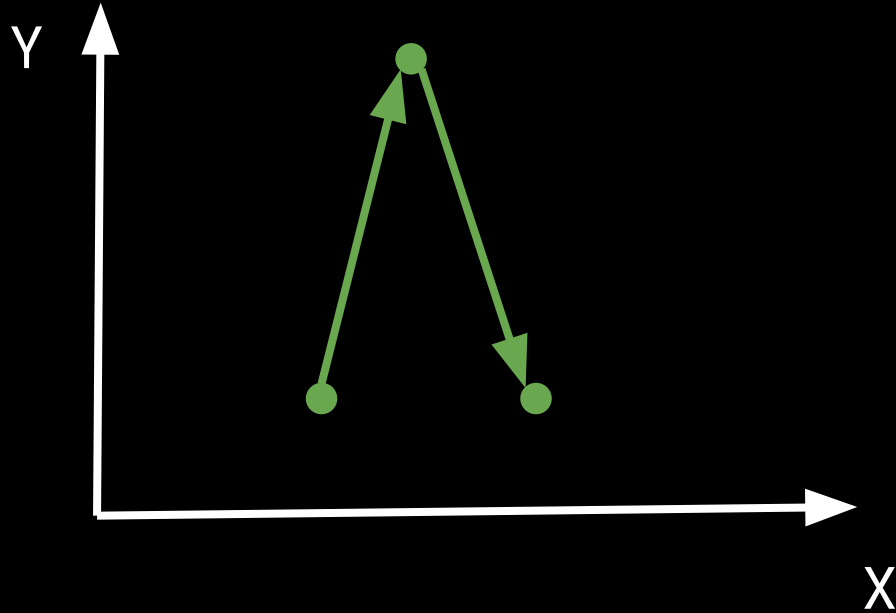
The Plan

The Almighty Tree

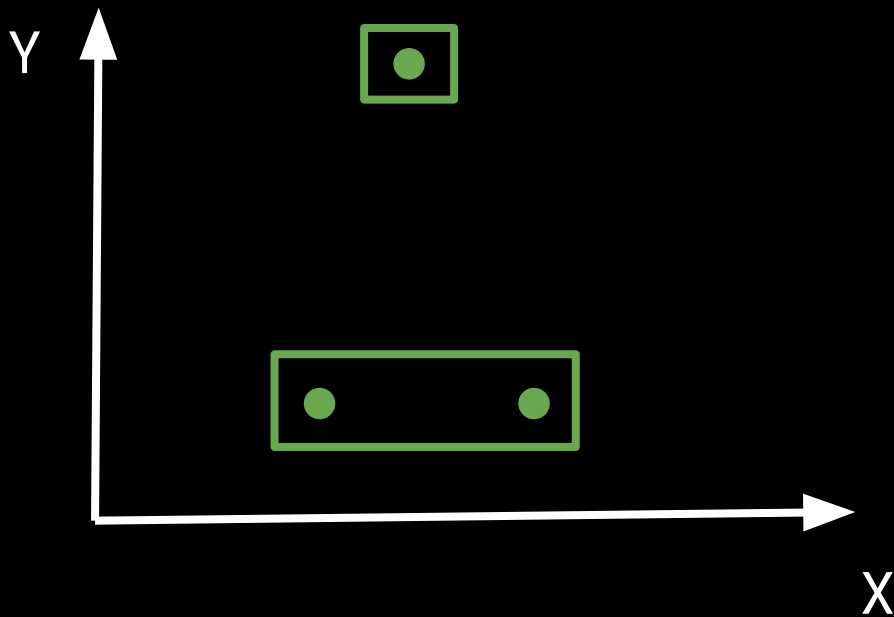
The Concept of Order

The Curse of Dimensionality

B-tree order vs. proximity



B-tree order vs. proximity



Coordinates: R-trees

**Vectors: inverted indexes, navigable
small world**

```
CREATE EXTENSION IF NOT EXISTS vector;  
DROP TABLE IF EXISTS t_vector_demo;  
CREATE TABLE t_vector_demo (  
    embedding vector(2)  
);  
-- 1M rows, 2D points  
INSERT INTO t_vector_demo (embedding)  
SELECT ARRAY[  
    random(),  
    random()  
]::vector  
FROM generate_series(1, 1_000_000);  
CREATE INDEX t_vector_demo_btree ON t_vector_demo (embedding);  
ANALYZE t_vector_demo;
```

```
EXPLAIN ANALYZE SELECT embedding FROM t_vector_demo ORDER BY embedding <-> '[0.5,0.5]' LIMIT
5;
Limit (cost=34515.64..34515.65 rows=5 width=21) (actual time=179.622..179.625 rows=5.00
loops=1)
  Buffers: shared hit=5406
    -> Sort (cost=34515.64..37015.64 rows=1000000 width=21) (actual time=179.620..179.622
rows=5.00 loops=1)
      Sort Key: ((embedding <-> '[0.5,0.5]')::vector))
      Sort Method: top-N heapsort Memory: 25kB
      Buffers: shared hit=5406
        -> Seq Scan on t_vector_demo (cost=0.00..17906.00 rows=1000000 width=21) (actual
time=0.023..119.924 rows=1000000.00 loops=1)
          Buffers: shared hit=5406
Planning Time: 0.078 ms
Execution Time: 179.659 ms
```

Hierarchical Navigable Small World

```
CREATE INDEX t_vector_demo_embedding_hnsw ON t_vector_demo  
USING hnsw (embedding vector_l2_ops);
```

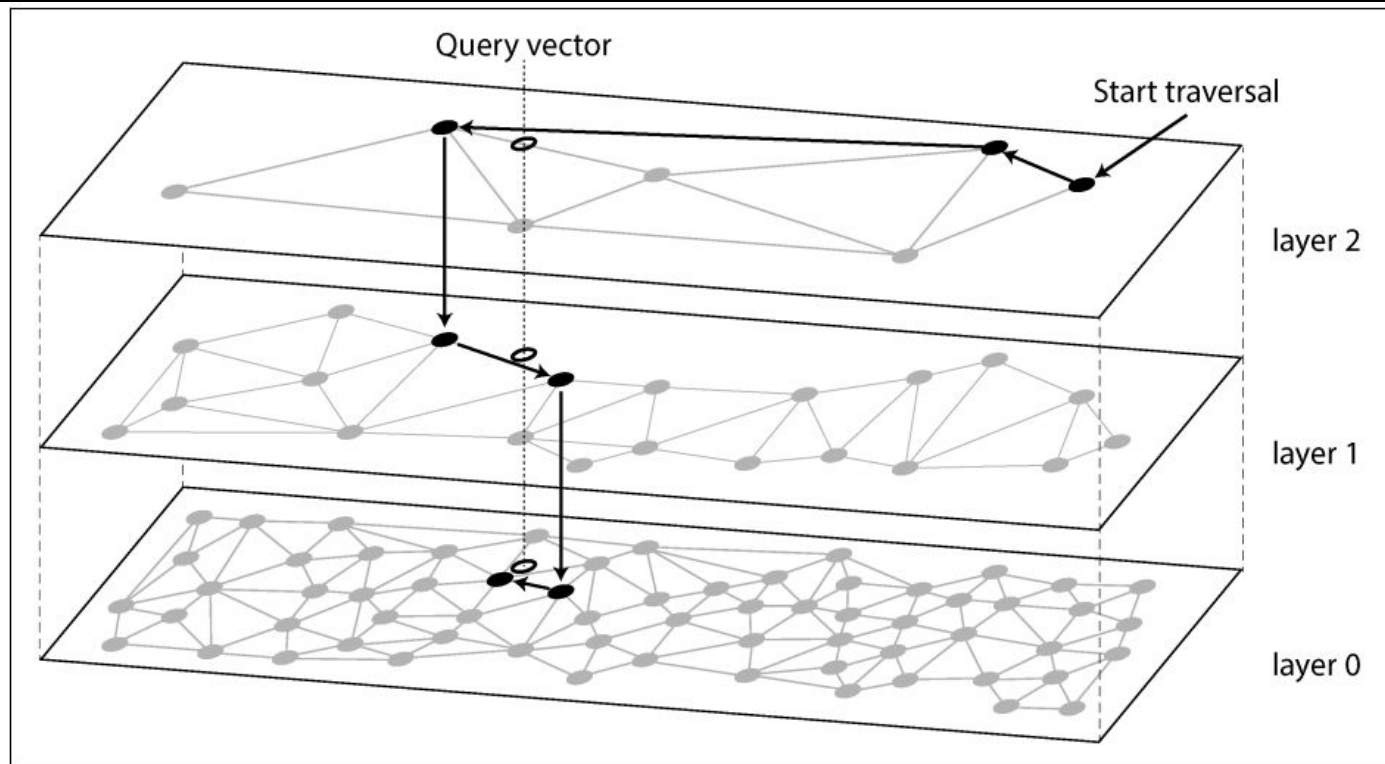


Figure 4-11. Searching for the database entry that is closest to a given query vector in an HNSW index

Source: Designing Data-Intensive Applications, 2nd edition, pp. 149

```
EXPLAIN ANALYZE SELECT embedding FROM t_vector_demo ORDER BY embedding <->
'[0.5,0.5]' LIMIT 5;
Limit (cost=128.94..129.90 rows=5 width=21) (actual time=0.308..0.314 rows=5.00
loops=1)
  Buffers: shared hit=284
    -> Index Scan using t_vector_demo_embedding_hnsw on t_vector_demo
(cost=128.94..192884.00 rows=1000000 width=21) (actual time=0.308..0.312 rows=5.00
loops=1)
      Order By: (embedding <-> '[0.5,0.5']::vector)
      Index Searches: 1
      Buffers: shared hit=284
Planning:
  Buffers: shared hit=1
Planning Time: 0.103 ms
Execution Time: 0.337 ms
```

Dissimilarity search: implementing in-memory vector search algorithms to PostgreSQL

• Jonathan S. Katz **AWS**



PGCONF.EU
2024



PostgreSQL
Europe

The Outcome

The Almighty Tree

The Concept of Order

The Curse of Dimensionality

Next talk

A tuple in flow

Bonus

To really *internalize* B-trees ...



Birch water
Some supermarkets
or [amazon.de](https://www.amazon.de)

Thank you !

